

Operating system – Suggestions and PYQ

PYQ-2024 & 2023

Section:A → 10*1=10

1. What is Kernel in OS?

Kernel is the core part of OS that manages hardware resources in a computer system , and it acts as an interface between hardware and user program.

2. What is a Process ?

Program in Execution is called a process.

3. What is Fragmentation ?

Fragmentation is the condition in which memory is wasted because free memory is divided into non contiguous divisions , reducing efficient memory utilization.

(You can give example of Internal and External fragmentation)

4. What is Bootstrapping in an OS?

Bootstrapping is the process of loading OS into main memory and starting its execution when the computer is powered on.

5. Virtual Memory is

- a. Larger than physical memory
- b. Faster than RAM
- c. Only available in UNIX OS
- d. None of the above

6. Which of the following scheduling algorithm could result in starvation?

- a. FCFS
- b. Shortest Job Next
- c. RR
- d. All of the above

7. Which of the following is not a type of OS?

- a. Batch
- b. Network
- c. Distributed
- d. Compiler based

8. In multiprogramming the number of processes in memory is called → Degree of multiprogramming
9. Which page replacement algorithm suffers from belady's anomaly?
- FIFO
 - LRU
 - Optimal
 - b and c
10. What is the Difference between User mode and Kernel Mode?

<i>User Mode</i>	<i>Kernel Mode</i>
<i>Runs user applications</i>	<i>Runs operating system kernel</i>
<i>Cannot execute privileged instructions</i>	<i>Can execute privileged instructions</i>

11. What is Process control block(PCB)?

A PCB is a data structure maintained by OS to store all the important information about the process.

It contains details like Process ID , Process state , Program counter , CPU register.

12. What is race condition?

A **race condition** occurs when **two or more processes or threads access shared data simultaneously and the final result depends on the order of their execution**, leading to incorrect or unpredictable results.

(You know the example!)

13. State necessary conditions for a deadlock to arise.

Mutual Exclusion

Hold and Wait

No pre-emption

Circular Wait

14. What is Internal and external fragmentation ?

Internal Fragmentation:

Occurs when **allocated memory is larger than the required memory**, leaving **unused space inside** the allocated block.

External Fragmentation:

Occurs when **free memory is scattered into small non-contiguous blocks**, so a **large contiguous memory allocation is not possible** even though total free memory is sufficient.

(You know the examples)

Section-B

1.

a) Define Operating System (OS)

An **Operating System (OS)** is system software that acts as an interface between the user and computer hardware and manages system resources like CPU, memory, and I/O devices.

Objectives of an OS:

- **Convenience** – makes the computer easy to use
- **Efficiency** – utilizes hardware resources effectively
- **Resource management** – manages CPU, memory, and devices
- **Security & protection** – prevents unauthorized access

b) Difference between Single User OS and Multiuser OS

Single User OS	Multiuser OS
Supports one user at a time	Supports multiple users simultaneously
Resources used by one user only	Resources shared among users
Simple design	More complex design
Example: MS-DOS	Example: UNIX

2.

a) Context Switching (3 Marks)

Context switching is the process of **saving the state of a currently running process and loading the state of another process**, allowing the CPU to switch between processes.

Significance of Context Switching:

- Enables **multitasking**
- Ensures **CPU sharing among processes**
- Improves **system responsiveness**

b) Different Attributes of a Process (3 Marks)

A process has several important attributes stored in the **Process Control Block (PCB)**:

- **Process ID (PID)** – unique identifier
- **Process State** – new, ready, running, waiting, terminated
- **Program Counter** – address of next instruction
- **CPU Registers** – current execution data
- **Memory Information** – code, data, stack details
- **Priority & Scheduling Information**

3. **Critical Section Problem (2 Marks)**

The **critical section problem** deals with **designing a mechanism so that no two processes execute their critical sections simultaneously**, ensuring **data consistency** when accessing shared resources.

Busy Waiting (2 Marks)

Busy waiting is a situation where a process **continuously checks for a condition (lock) in a loop while waiting**, wasting CPU time.

Overcoming Busy Waiting using Semaphore Operations (2 Marks)

Busy waiting can be avoided by using **blocking semaphore operations**:

- **wait (P)**: If semaphore value ≤ 0 , the process is **blocked and put into a waiting queue**.
- **signal (V)**: Wakes up a **blocked process** instead of making it spin.
Thus, the process **sleeps instead of busy waiting**, saving CPU time.

4. **CPU Scheduling Algorithm – Numerical** **Waiting Time:**

The **total time a process spends waiting in the ready queue** for CPU execution.

Waiting Time = Turnaround Time – Burst Time

Turnaround Time:

The **total time taken from process arrival to its completion**, including waiting and execution time.

Turnaround Time = Completion Time – Arrival Time

5. Difference between internal and external fragmentation.

Internal Fragmentation

*Unused memory inside
allocated blocks*

*Occurs due to fixed-
size memory allocation*

External Fragmentation

*Unused memory
between allocated
blocks*

*Occurs due to variable-
size memory allocation*

Best fit and worst fit – Numerical

6.

Resource Allocation Graph (RAG)

A Resource Allocation Graph is a directed graph used by the operating system to represent the allocation of resources to processes and to analyze deadlock situations.

- Processes are represented by circles ($P_1, P_2, \dots$)
- Resources are represented by rectangles ($R_1, R_2, \dots$)
- Request edge ($P \rightarrow R$) shows a process requesting a resource
- Assignment edge ($R \rightarrow P$) shows a resource allocated to a process

Significance:

- Helps in deadlock detection and prevention
- A cycle in the graph indicates deadlock (if each resource has one instance)

Numerical on Logical address space and physical address space.

7. EMAT Numerical on paging

Virtual Memory :

Virtual Memory is a memory management technique that allows a system to execute programs larger than physical RAM by using secondary storage (disk) as an extension of main memory.

SECTION-C

1. a) What is paging ? Explain the paging methods with suitable diagram.

Paging

Paging is a memory management technique in which logical memory is divided into fixed-size blocks called pages and physical memory is divided into frames of the same size.

It eliminates external fragmentation.

Paging Method (Working)

1. A process is divided into pages (Page 0, Page 1, Page 2 ...).
2. Main memory is divided into frames (Frame 0, Frame 1, Frame 2 ...).
3. Pages are stored in any available frames (non-contiguous).
4. A page table maps each page number to a frame number.
5. Logical address = Page number + Offset
6. Physical address = Frame number + Offset

Paging Diagram -

Advantages of Paging

- Eliminates external fragmentation
- Efficient memory utilization
- Supports virtual memory

b) What are advantages of TLB with paging over paging technique?

Advantages of TLB with Paging over Simple Paging

- Faster memory access by reducing page table lookup time
- Improves CPU performance by avoiding frequent memory accesses
- Stores frequently used page table entries for quick access
- Reduces effective memory access time (EMAT)

c) What is demand paging?

Demand Paging is a virtual memory technique in which a page is loaded into main memory only when it is actually required, not in advance.

2. Define the term deadlock.

Explain various necessary conditions for a deadlock to occur.

Explain in brief about deadlock prevention.

Deadlock (2 Marks)

Deadlock is a situation in which two or more processes are permanently blocked, each waiting for a resource held by another process in the set.

Necessary Conditions for Deadlock (5 Marks)

Deadlock can occur only if all four conditions hold simultaneously (Coffman conditions):

1. Mutual Exclusion

At least one resource must be non-shareable.

2. Hold and Wait

A process holding at least one resource is waiting for additional resources.

3. No Preemption

Resources cannot be forcibly taken from a process.

4. Circular Wait

A circular chain of processes exists, each waiting for a resource held by the next.

Deadlock Prevention (3 Marks)

Deadlock prevention ensures that at least one necessary condition never holds:

- Mutual Exclusion → Make resources sharable where possible**
- Hold and Wait → Require processes to request all resources at once**
- No Pre-emption → Pre-empt resources when a process cannot get all it needs**
- Circular Wait → Impose resource ordering**

Thus, deadlock is prevented by breaking one of the four conditions.

3. What is the need of page replacement ?

Page replacement Numerical.

Which algorithm gives minimum number of page faults ?

Need of Page Replacement

Page replacement is needed when a page fault occurs and there is no free frame available in main memory.

The OS must select a page to remove (replace) so that the required page can be loaded.

Algorithm with Minimum Page Faults

The Optimal Page Replacement Algorithm gives the minimum number of page faults because it replaces the page that will not be used for the longest time in the future.

4. Numerical on Resource request algorithm Differentiate SCAN and CSCAN.

SCAN

Disk arm moves in both directions

Serves requests while moving forward and backward

Uneven waiting time

Also called Elevator algorithm

C-SCAN

Disk arm moves in one direction only

Serves requests only in one direction

More uniform waiting time

Also called Circular SCAN

5. Explain Peterson's solution for 2-Process critical section problem. Explain wait() and Signal() operations.

Peterson's Solution from Gate wallah slides.

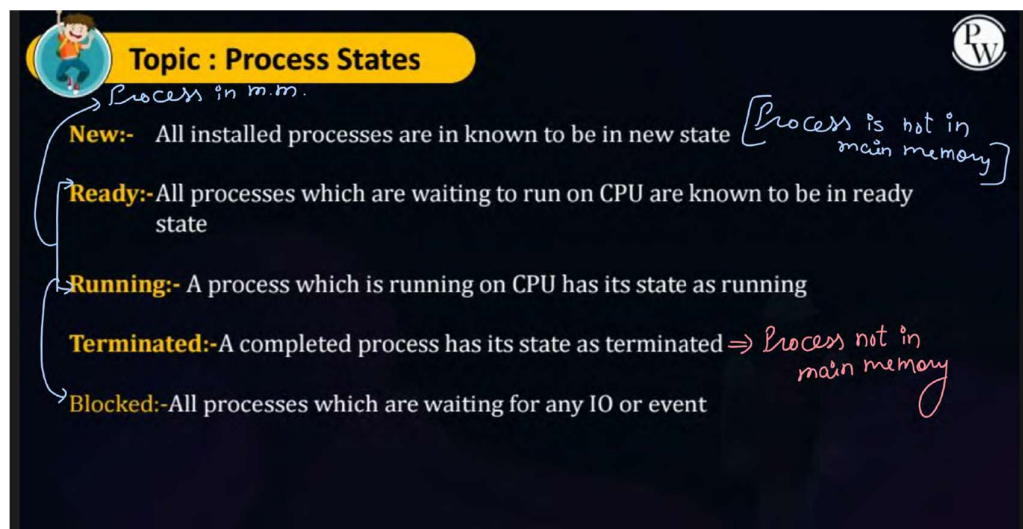
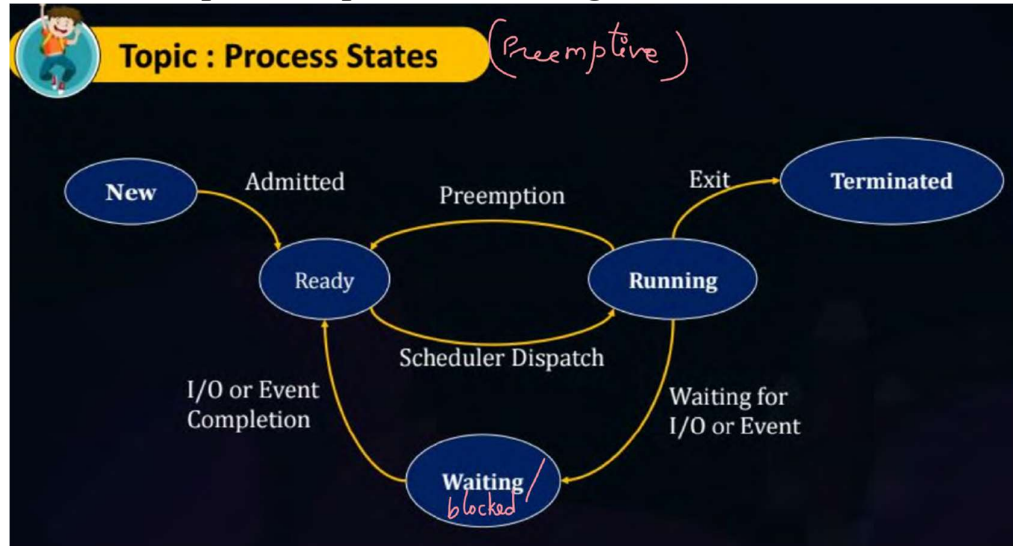
wait() and signal() Operations (Semaphore)

- **wait (P)**
Decrements semaphore value.
If value < 0 , the process is blocked.
 - **signal (V)**
Increments semaphore value.
If value ≤ 0 , a blocked process is awakened.
- Purpose**
- Used to control access to critical section
 - Prevents race condition and busy waiting

6. Numerical on disk scheduling algo.

PYQ-2023 – Long answers

1. Draw and explain the process state diagram.



2. Explain the difference between long term , short term , medium term scheduler.

Basis	Long-Term Scheduler (Job Scheduler)	Short-Term Scheduler (CPU Scheduler)	Medium-Term Scheduler
Purpose	Selects jobs/processes from disk and admits them into main memory	Selects one process from ready queue to allocate CPU	Suspends and resumes processes to manage memory and load
Level of Control	Controls degree of multiprogramming	Controls CPU time allocation	Controls memory usage and process mix

Speed of Execution	Slow (invoked infrequently)	Very fast (invoked very frequently)	Moderate speed
Main Function	Moves process from New → Ready	Moves process from Ready → Running	Moves process between Ready/Blocked ↔ Suspended
Memory Involvement	Brings process into memory	No memory management role	Swaps processes in/out of memory
Examples	Batch systems	Time-sharing systems	Systems with swapping/virtual memory

3. What is Critical Section?

What requirement should be satisfied for a solution to the critical section problem?

Critical Section :

A **critical section** is a part of a program where a process/thread accesses **shared data or shared resources** (such as variables, files, or memory).

To avoid **race conditions**, only **one process at a time** is allowed to execute its critical section.

Requirements for a Solution to the Critical Section Problem :

A correct solution must satisfy the following **three conditions**:

Mutual Exclusion

At most **one process** can be inside its critical section at any time.

Progress

If no process is in the critical section and some processes want to enter, the selection of the next process **cannot be postponed indefinitely**.

Bounded Waiting

There must be a **limit** on how many times other processes can enter the critical section after a process has made a request, preventing **starvation**.

4. Discuss deadlock detection algorithm with an example.

Deadlock Detection Algorithm

A deadlock detection algorithm is used by the operating system to check whether a deadlock has already occurred. Unlike deadlock prevention or avoidance, the system allows deadlocks to happen and then detects them.

Deadlock Detection Algorithm (Single Instance of Each Resource)

Algorithm

- Construct a Wait-For Graph (WFG):
 - Each node represents a process.
 - A directed edge $P_i \rightarrow P_j$ exists if process P_i is waiting for a resource currently held by P_j .
- Check for cycles in the Wait-For Graph:
 - Cycle present → Deadlock exists
 - No cycle → No deadlock

Example

Consider three processes and three resources:

- Processes: P1, P2, P3
- Resources: R1, R2, R3 (single instance each)

Process	Holds	Requests
P1	R1	R2
P2	R2	R3
P3	R3	R1

Wait-For Graph

- P1 → P2 (P1 waits for R2 held by P2)
- P2 → P3 (P2 waits for R3 held by P3)
- P3 → P1 (P3 waits for R1 held by P1)

This forms a cycle:

P1 → P2 → P3 → P1

Conclusion: A cycle exists, so the system is in a deadlock.

Deadlock Detection Algorithm (Multiple Instances of Resources)

When resources have multiple instances, the OS uses a matrix-based detection algorithm, similar to Banker's algorithm.

Data Structures Used

- Available – Vector of available resource instances
- Allocation – Resources currently allocated to processes
- Request – Resources requested by processes

Steps (Brief)

- Initialize Work = Available
 - Find a process whose Request ≤ Work
- If found, release its resources and update Work
 - Repeat until no such process exists
 - Processes that cannot finish are deadlocked

5. Explain various file allocation methods.

File Allocation Methods

In an operating system, file allocation decides how files are stored in disk blocks. There are three primary methods:

1. Contiguous Allocation

- Definition: Each file occupies a set of contiguous disk blocks.
- Advantages:
 - Simple and fast access (direct access possible).
 - Minimal disk head movement for sequential access.
- Disadvantages:
 - External fragmentation occurs.
 - Hard to expand files dynamically.
- Example: File occupies blocks 10–15 consecutively.

2. Linked Allocation

- Definition: Each file is a linked list of disk blocks; each block contains a pointer to the next block.
- Advantages:
 - No external fragmentation.
 - Easy file expansion.
- Disadvantages:
 - Random access is slow (must traverse the list).
 - Pointer storage uses extra space.

- Example:
- Block 5 → Block 8 → Block 12 → Block 20 → NULL

3. Indexed Allocation

- **Definition:** Each file has an index block containing pointers to all its disk blocks.
- **Advantages:**
 - Supports direct access efficiently.
 - Solves external fragmentation.
- **Disadvantages:**
 - Overhead of index block.
 - Index block may become large for very big files.
- **Example:**

Index Block: [5, 8, 12, 20] → Points to actual data blocks

Sequential access → Linked allocation

Direct access → Indexed allocation

Fast and simple → Contiguous allocation

SECTION-C

1. Explain in details various types of OS.

1. Uniprogramming OS

- **Definition:** Only **one process** runs at a time.
- **Use:** Early computers.
- **Feature:** Simple, no multitasking, CPU idle during I/O.

2. Multiprogramming OS

- **Definition:** Allows **multiple processes** in memory; CPU switches to another when one waits for I/O.
- **Use:** Mainframes and batch processing systems.
- **Feature:** Increases **CPU utilization** and system throughput.

3. Multitasking OS (Time-Sharing OS)

- **Definition:** Multiple tasks **share CPU in small time slices**; users feel simultaneous execution.
- **Use:** Desktop systems (Windows, Linux, macOS).
- **Feature:** Quick **response time**, supports **interactive use**.

4. Multiprocessing OS

- **Definition:** Uses **multiple CPUs** to execute multiple processes **simultaneously**.
- **Use:** High-performance servers, supercomputers.
- **Feature:** **Parallel processing**, higher throughput, better reliability.

5. Multiuser OS

- **Definition:** Multiple users can **use the system simultaneously**.
- **Use:** UNIX, Linux servers.
- **Feature:** Provides **user isolation**, resource sharing, and security.

6. Real-Time OS (RTOS)

- **Definition:** Provides **guaranteed response time** for critical tasks.
- **Use:** Airplanes, medical devices, industrial automation.
- **Feature:** Deterministic, fast, and **reliable**.

7. Embedded OS

- **Definition:** Special OS designed for **embedded systems** like microwaves, washing machines.
- **Use:** IoT devices, consumer electronics.
- **Feature:** Lightweight, efficient, minimal resource usage.

8. Handheld Device OS

- **Definition:** OS for **mobile devices, smartphones, PDAs**.
- **Use:** Android, iOS, Windows Mobile.
- **Feature:** Optimized for **battery life, touch input, small memory**.

2. What are semaphores?

Explain 2 primitive semaphore operations?

State dining philosophers problem and give a solution using semaphores.

What are Semaphores?

- A **semaphore** is a **synchronization tool** used in operating systems to **control access to shared resources** by multiple processes.
- It is an **integer variable** that is **modified using two atomic operations**.
- Helps to **solve critical section problems** and **prevent race conditions**.

Two Primitive Semaphore Operations

A semaphore supports **two atomic operations**:

Operation	Description
-----------	-------------

**Wait (P /
down)**

Decreases the semaphore value by 1. If the result is **negative**, the process is **blocked**.

**Signal (V /
up)**

Increases the semaphore value by 1. If **any process is waiting**, one of them is **unblocked**.

Pseudo-code:

wait(S):

$S = S - 1$

 if $S < 0$:

 block the process

signal(S):

$S = S + 1$

 if $S \leq 0$:

 wake up a waiting process

Dining Philosophers Problem

Problem Statement:

- Five philosophers sit around a table.
 - Each philosopher **thinks or eats**.
 - There are **5 forks**; each philosopher needs **two forks (left & right)** to eat.
 - **Goal:** Prevent **deadlock** and **starvation**.
-

Solution Using Semaphores

What is page fault?

A **page fault** occurs in a computer system when a program tries to access a **page** (a fixed-size block of memory) that is **not currently present in the main memory (RAM)**.